

PREPARACIÓN DEL ENTORNO

TÉCNICO SUPERIOR UNIVERSITARIO DESARROLLO DE SOFTWARE

PRESENTA:

BUSTILLOS MOLINA PALOMA JIMENA

CORONEL LIMIAS HECTOR

CRUZ VERDE DANIEL MICHEL

PEREZ BARRIOS ERNESTO

RODRÍGUEZ GÓMEZ MARÍA BELÉN

PROFESOR:

HECTOR SALDAÑA BENITEZ

SAN JUAN DEL RIO, QRO.

SEPTIEMBRE DE 2026

Índice

1. INTRODUCCIÓN	4
2. ELECCIÓN Y JUSTIFICACIÓN DEL WORKFLOW	4
Diagrama	6
3. PARÁMETROS DE CONFIGURACIÓN	7
Plantilla de Pull Request.....	8
Convención de commits	10
Versionado semántico y número de compilación.....	10
CONTRIBUTING.mdFlujo de trabajo.....	11
4. INTEGRACIÓN CONTINUA (CI).....	12
Flujo de validación del PR	12
5. ESTRATEGIA DE DESPLIEGUE Y DISTRIBUCIÓN MÓVIL	12
Distribución elegida	13
Métricas de estabilidad.....	14
Criterios de reversión y hotfix.....	15
6. PLAN MÍNIMO DE PRUEBAS	16
Pruebas unitarias.....	16
Verificación post-distribución.....	16
10. Conclusiones.....	18
11. Fuentes de consulta.....	18

Anexo A. Relación con documentos previos.....	19
---	----



1. INTRODUCCIÓN

FutSchool es una aplicación móvil orientada a la organización de torneos de fútbol escolares. Este documento define la propuesta de trabajo para administrar los cambios del proyecto, revisar su integración y distribuir versiones de prueba.

Se establecen el workflow, las reglas del repositorio, las convenciones de commits, el versionado, la estrategia de distribución y el plan mínimo de pruebas. Su implementación será progresiva, conforme se incorpore el código de la aplicación.

2. ELECCIÓN Y JUSTIFICACIÓN DEL WORKFLOW

Para FutSchool se selecciona **GitHub Flow**, un flujo basado en ramas temporales y Pull Requests hacia una rama principal. Este proceso permite desarrollar cambios por separado y revisarlos antes de incorporarlos al trabajo compartido.

Se consideraron las siguientes alternativas:

Workflow	Característica principal	Consideración para FutSchool
GitHub Flow	Ramas temporales y Pull Requests hacia una ramaprincipal.	Permite trabajar por funcionalidades con pocos pasos y una revisión clara.
Gitflow	Ramas específicas para desarrollo, funcionalidades, versiones y correcciones.	Es útil cuando se necesita organizar distintos ciclos de lanzamiento. Para el alcance actual, implicaría administrar más ramas y reglas.
Trunk-Based	Integración frecuente de cambios pequeños en una rama principal.	Requiere que el equipo pueda revisar e integrar continuamente. Resulta más exigente si los integrantes avanzan en momentos distintos o todavía están aprendiendo el proceso.

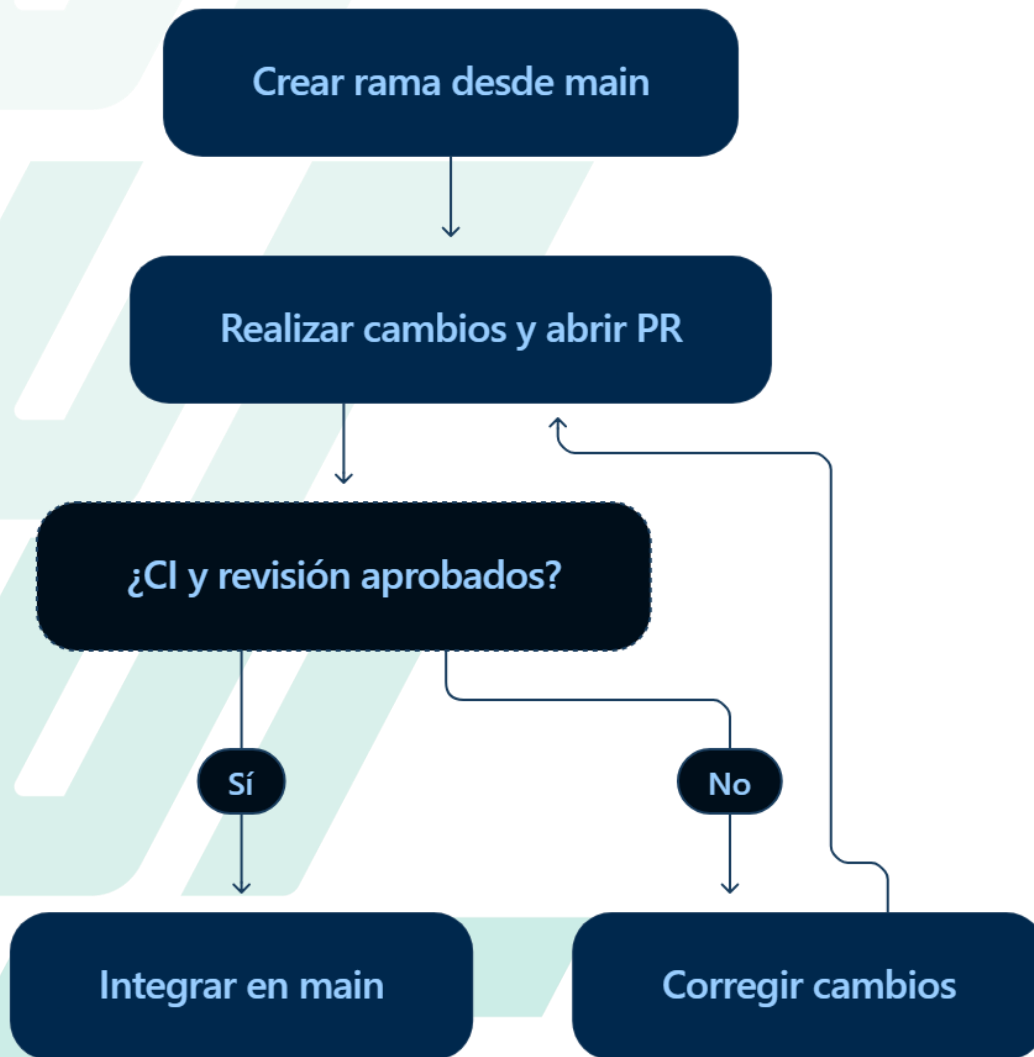
GitHub Flow se adapta al equipo porque establece un procedimiento sencillo sin requerir una organización compleja. Cada integrante puede crear una rama según la actividad que vaya a realizar y solicitar una revisión cuando el cambio esté listo.

La rama **main** concentra los cambios aceptados. Las ramas temporales permiten trabajar sin modificar directamente esa versión, mientras que los PR facilitan explicar lo realizado, recibir observaciones y corregir problemas antes de integrar.

Esta elección también permite comenzar con documentación y aplicar el mismo proceso cuando se incorpore el desarrollo de la aplicación. No se necesita una rama permanente `develop`, ya que las actividades se integrarán directamente a `main` mediante PR.

Diagrama

El diagrama representa la creación de una rama desde main, la realización de cambios, la apertura del PR y su validación. Si existen observaciones o comprobaciones fallidas, los cambios se corrigen en la misma rama y se vuelven a revisar.



Cuando el PR cumple las condiciones, se integra en main y se elimina la rama temporal. Los cambios permanecen en la rama principal.

3. Parámetros de configuración

Las siguientes reglas se proponen para mantener una revisión básica y evitar que se incorporen cambios sin verificar:

Parámetro	Configuración propuesta	Justificación
Rama protegida	Proteger main y exigir PR para incorporar cambios.	Evitar modificaciones directas sin revisión.
Revisiones	Requerir la aprobación de otro integrante y resolver sus observaciones	Incorporar una segunda revisión del trabajo.
CI	Exigir que pasen las comprobaciones disponibles antes del merge.	Evitar integrar cambios con problemas sin atender.
Plantilla de PR	Incluir tarea relacionada, descripción de cambios, forma de verificación y evidencia cuando corresponda.	Comprobar que se cumplen las validaciones disponibles.
Commits	Utilizar feat: , fix: y docs: , seguidos de una descripción breve.	Identificar el propósito de cada rama.
Versionado	Utilizar vX.Y.Z (build) , por ejemplo, v0.1.0 (1) .	Facilitar su revisión y seguimiento.

Plantilla de Pull Request

Se utilizará una plantilla en “.github/pull_request_template.md” con los siguientes campos:

- Tarea o historia relacionada: identificador o enlace de la actividad.
- Descripción: qué se cambió y por qué.
- Verificación realizada: cómo se revisó el cambio y qué resultado se obtuvo.
- Evidencia: capturas o resultados cuando corresponda.
- Lista de comprobación: cambios revisados, documentación actualizada cuando aplique y comprobaciones automáticas aprobadas.

La plantilla permite que el revisor conozca el objetivo del cambio y cuente con información suficiente para evaluarlo.

Archivo: “.github/pull_request_template.md”

##Contexto

- Módulo / Pantalla:
- Ticket / Issue: #
- Tipo de cambio: [Feature | Bugfix | Chore | UI Test]

##Checklist de calidad

- La aplicación compila sin advertencias críticas - ☐ La aplicación compila sin advertencias críticas
- Las pruebas unitarias pasaron en el CI - ☐ Las pruebas unitarias pasaron en el CI
- No se introducen dependencias nativas sin justificación - ☐ No se introducen dependencias nativas sin justificación
- Formato y convención de commits respetados - ☐ Formato y convención de commits respetados
- Revisado por al menos un compañero de equipo - ☐ Revisado por al menos un compañero de equipo

Como mejora opcional se puede agregar .github/CODEOWNERS para solicitar revisiones específicas en carpetas de configuración nativa.



Convención de commits

Los commits utilizarán un tipo seguido de una descripción breve:

Tipo	Uso	Ejemplo
feat	Nueva funcionalidad	feat: agregar registro de equipos
fix	Corrección de errores	fix: corregir validación de partidos
docs	Cambios de documentación	docs: actualizar guía de contribución
refactor	Reestructuración sin cambio funcional	refactor: separar servicio de torneos
test	Pruebas	test: agregar casos del validador
chore	Mantenimiento del proyecto	chore: actualizar dependencias

Esta convención permite reconocer el propósito de los cambios al consultar el historial.

Versionado semántico y número de compilación

Las entregas se identificarán con el formato vX.Y.Z (build N). La versión permitirá distinguir los cambios funcionales y el build identificará cada compilación distribuida.

En el versionado semántico, X representa cambios mayores incompatibles, Y nuevas funcionalidades compatibles y Z correcciones compatibles. Las versiones 0.x.y corresponden al desarrollo inicial y todavía pueden presentar cambios importantes.

Ejemplo	Interpretación
v0.1.0 (build 1)	Primera entrega de prueba.
v0.2.0 (build 2)	Entrega con nuevas funcionalidades.
v0.2.1 (build 3)	Corrección de errores de la entrega anterior.
v1.0.0 (build 20)	Primera versión estable del producto.

El número de build aumentará con cada compilación distribuida, aunque se mantenga la misma versión funcional.

CONTRIBUTING.md

Flujo de trabajo

Seguimos GitHub Flow:

1. Crear una rama desde main: feature/nombre-modulo, fix/descripcion o chore/descripcion
2. Realizar cambios pequeños y commits con Conventional Commits.
3. Abrir un Pull Request hacia main.
4. Esperar CI en verde y al menos una aprobación.
5. Resolver observaciones y realizar el merge.
6. Eliminar la rama temporal después del merge.

Versionado

Se emplea Semantic Versioning con formato vX.Y.Z y número de compilación: vX.Y.Z (build N).

Calidad

Toda funcionalidad debe incluir o actualizar pruebas cuando corresponda y debe cumplir los criterios de aceptación de la historia de usuario.

4. INTEGRACIÓN CONTINUA (CI)

Se utilizará GitHub Actions en los PR dirigidos a `main` y en los push realizados sobre esa rama. Mientras el repositorio contenga únicamente documentación, la validación inicial comprobará la existencia de `README.md`. Esta comprobación permite verificar el funcionamiento del workflow, pero no representa una prueba de la aplicación.

Cuando se incorpore Flutter, el CI se ampliará para instalar dependencias, analizar el código y ejecutar pruebas unitarias. Las comprobaciones obligatorias deberán aprobarse antes del merge.

El objetivo mínimo es instalar dependencias y ejecutar las pruebas unitarias. El Plan DevOps ya contempla GitHub Actions para Build/Test y exige que el PR pase los checks antes del merge.

Flujo de validación del PR

1. Crear feature/login-validation desde main.
2. Implementar la función pura y su prueba unitaria.
3. Hacer push de la rama.
4. Abrir PR hacia main.
5. GitHub Actions instala dependencias y ejecuta pruebas.
6. Si el check falla, el merge queda bloqueado hasta corregir.
7. Si el check pasa y existe una aprobación, el PR puede integrarse.

5. ESTRATEGIA DE DESPLIEGUE Y DISTRIBUCIÓN MÓVIL

Nivel	Qué se valida	Ejemplo en FutSchool
Unitarias	Funciones puras y utilidades	Validador/formateador de formulario
Componente	Comportamiento de una pantalla o componente	Login: campos, mensajes y estado
Integración/flujo	Interacción entre módulos	Registro → equipo → torneo
Post-distribución	Funcionamiento de versión	Inicio de sesión, torneos,

	candidata	partidos y notificaciones
--	-----------	---------------------------

Ejemplo de función pura y prueba

Función:

validateEmail(email) → true/false

Casos mínimos:

- correo válido → true
- correo vacío → false
- correo sin dominio → false

La prueba debe ejecutarse automáticamente dentro del CI para cada PR hacia main.

Criterios de entrada: funcionalidad implementada, historia y criterios de aceptación definidos y funcionalidad ejecutable. Criterios de salida: resultado esperado, errores corregidos y funcionalidades relacionadas conservadas.

7. Estrategia de despliegue y distribución móvil

Distribución elegida

Se propone una distribución interna mediante **Firebase App Distribution**, dirigida inicialmente al equipo y evaluadores. Esta herramienta permite compartir compilaciones con personas de prueba y recopilar comentarios antes de una publicación más amplia.

Esta estrategia corresponde al alcance actual de FutSchool porque permite revisar cada entrega con un grupo limitado y corregir problemas antes de continuar. La publicación en tiendas se considera una etapa posterior.

El proceso propuesto será:

1. Preparar una compilación identificada con versión y build.
2. Ejecutar las pruebas disponibles.

3. Distribuir la al grupo de prueba.
4. Verificar su instalación y las funciones incluidas.

Registrar errores y decidir si la entrega puede continuar o necesita una corrección.

5.

Métricas de estabilidad

Se propone observar cada build durante una ventana inicial de 24 horas, incluyendo una ronda de pruebas del equipo. El periodo por sí solo no será suficiente: deberán existir pruebas ejecutadas y datos registrados.

Metriica	Medición propuesta	Criterio inicial propuesto
Usuarios sin cierres inesperados — crash-free users—	Porcentaje de usuarios que no experimentaron cierres inesperados durante el periodo observado. Se propone utilizar Crashlytics cuando se integre.	Mantener al menos 99.5%. Cualquier cierre reproducible deberá revisarse antes de continuar.
Errores de red	Solicitudes fallidas por conexión o tiempo de espera agotado, divididas entre las solicitudes realizadas, multiplicadas por 100. Se registrarán desde la aplicación.	Investigar y detener el avance si superan el 5% durante las pruebas con conectividad disponible.
Fallas funcionales críticas	Registro de errores que impidan completar una función incluida en la entrega.	No aceptar fallas críticas pendientes.

Los porcentajes son criterios propuestos para la práctica y podrán ajustarse con los primeros resultados. Se registrarán también las cantidades de usuarios y solicitudes, porque un porcentaje aislado puede resultar poco representativo con un grupo pequeño.

Las pruebas intencionales sin conexión se registrarán por separado. Si todavía no existe instrumentación o no hay datos suficientes, la métrica se reportará como pendiente de evaluación

Criterios de reversión y hotfix

Se detendrá la distribución de una build cuando presente cierres reproducibles, errores que impidan completar funciones principales o resultados que incumplan los criterios establecidos.

En una aplicación móvil, detener la distribución no elimina la versión ya instalada en los dispositivos. Por ello, la respuesta prevista será identificar el cambio que provocó el problema y preparar una nueva compilación.

Situación	Acción propuesta
Error detectado antes de distribuir	Corregir y repetir las pruebas
Falla critica despues de distribuir	Detener nuevas entregar de esa build y registrar la incidencia.
Cambio reciente que pueda revertirse de forma segura	Revertirlo en el código, revisar compatibilidad y generar una nueva build
Error que requiera una corrección específica	Preparar un hotfix mediante una rama fix/, PR y validaciones.
La compilación corregida tendrá un nuevo número de build y deberá pasar nuevamente las pruebas unitarias y la verificación post-distribución.	

6. PLAN MÍNIMO DE PRUEBAS

El plan contempla pruebas unitarias de utilidades y una revisión de la compilación distribuida. Se aplicarán a las funcionalidades disponibles en cada entrega.

Pruebas unitarias

Se verificarán funciones pequeñas con entradas y resultados definidos, como validaciones y formateadores. Esto permite detectar errores sin ejecutar todo el recorrido de la aplicación.

Como caso inicial se propone un validador de correo:

Entrada	Resultado esperado
Correo con formato valido	Aceptado
Campo vacío	Rechazado
Correo sin dominio	Rechazado

Cuando exista la implementación en Flutter, estas pruebas se ejecutarán con `flutter test` dentro del CI. Un resultado fallido deberá corregirse antes de integrar el PR.

Verificación post-distribución

Después de compartir una build se comprobará:

Verificación	Resultado esperado
Instalación y apertura	La aplicación se instala y abre sin cierres inesperados.
Version y build	Coinciden con la entrega que se está evaluando.
Funcionalidad incluida	Cumple sus criterios de aceptación.
Navegación relacionada	Permite completar el recorrido sin bloqueos.
Manejo de errores de conexión	Informa del problema sin cerrar inesperadamente la aplicación.

Se registrarán la versión, el dispositivo, el caso probado, el resultado obtenido y las incidencias. La entrega se considerará aceptable cuando las pruebas previstas estén completas y no existan fallas críticas pendientes.



7. Conclusiones

La preparación del entorno de FutSchool queda alineada con el trabajo definido en Sprint 0 y el Plan DevOps: GitHub como repositorio y gestión del código, GitHub Projects para actividades, GitHub Flow para integración, GitHub Actions para CI, pruebas progresivas y Firebase App Distribution/tiendas para distribución. La práctica formaliza además las reglas de protección de main, la plantilla de PR, Conventional Commits, Semantic Versioning, criterios de rollout y un mecanismo de contingencia.

El resultado esperado es que cada cambio pueda rastrearse desde una historia o actividad hasta su Pull Request, validación automática, revisión y eventual distribución. Las decisiones de despliegue deberán acompañarse de métricas y criterios explícitos para detener el avance o aplicar una corrección.

8. Fuentes de consulta

- Git SCM — Branching Workflows: <https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows>
- GitHub Docs — GitHub Flow / Pull Requests: <https://docs.github.com/>
- Atlassian — Gitflow Workflow: <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>
- Trunk Based Development: <https://trunkbaseddevelopment.com/>
- Kubernetes — Deployments / Rolling Updates: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- AWS — Blue/Green Deployments: <https://docs.aws.amazon.com/whitepapers/latest/overview-deployment-options/bluegreen-deployments.html>
- Google SRE Workbook — Canarying Releases: <https://sre.google/workbook/canarying-releases/>
- Unleash Documentation — Feature Flags: <https://docs.getunleash.io/>
- Statsig Documentation — A/B Testing: <https://docs.statsig.com/>
- Documentos base del proyecto: Board/Sprint 0, Plan DevOps y Matriz y Dictamen de Comunicación.

Anexo A. Relación con documentos previos

El Board/Sprint 0 establece el repositorio, GitHub Flow, Board, backlog, CI y comunicación como actividades de preparación. El Plan DevOps amplía esa planeación con arquitectura, ciclo DevOps, versionamiento, CI/CD, pruebas, despliegue, monitoreo y riesgos. La Matriz de Comunicación define Microsoft Teams y sus canales como complemento de GitHub Projects. Este documento integra esos elementos específicamente para cumplir los entregables de la práctica de investigación.

Referencias internas: Board/Sprint 0: tablero, backlog, criterios de aceptación y Sprint 0. Plan DevOps: caso FutSchool, ciclo DevOps, versionamiento, CI/CD, pruebas, despliegue y monitoreo. Matriz de Comunicación: herramienta y canales de comunicación.